2.1.2 Longest common subsequence (LCS)

Consideriamo ora il problema della *più lunga sottosequenza comune* (*longest common subsequence* o *LCS*). Una *sottosequenza* di una sequenza s è una sequenza di elementi di s presi nello stesso ordine, ossia è individuata da una sequenza di indici di s.

Problema: date due sequenze trovare una sottosequenza comune di lunghezza massima (individuata da una sequenza di coppie di indici). Esempi reali di tale problema si trovano in biologia (trovare la più lunga sottosequenza comune a due sequenze di DNA), nell'ambito della sicurezza informatica (individuare, in un log costituito da una sequenza di comandi, le sottosequenze che indicano la presenza di un possibile attacco al sistema), etc.

L'algoritmo ingenuo ("brute force"), date le due sequenze s_1 ed s_2, di lunghezza m ed n rispettivamente, genera tutte le sottosequenze di s_1, e per ognuna di esse controlla se questa è anche sottosequenza di s_2, tenendo traccia della più lunga trovata. Ogni sottosequenza di s_1 corrisponde a un sottoinsieme degli indici di s_1, quindi esse sono 2^m. Per ognuna occorrono nel caso peggiore n passi per controllare se è anche sottosequenza di s_2, utilizzando una scansione simultanea della sottosequenza scelta e di s_2 (l'algoritmo è simile al passo *merge* di mergesort: si avanza su s_2 finché non si trova un elemento uguale al prossimo della sottosequenza, quindi si avanza su entrambe...). La complessità è quindi $n \times 2^m$.

Una formulazione induttiva della soluzione può essere data nel modo seguente, dove $X[1..m]$ e $Y[1..n]$ sono le due sequenze e $LCS(i, j)$ indica una sottosequenza comune di lunghezza massima tra i prefissi $X[1..i]$ e $Y[1..j]$.

$LCS(0, j) = [\,]$ per $0 \leq j \leq n$
$LCS(i, 0) = [\,]$ per $0 \leq i \leq m$
per $i \neq 0$, $j \neq 0$:
$LCS(i, j) = LCS(i-1, j-1) \cdot (i, j)$ se $X(i) = Y(j)$
$LCS(i, j) = max(LCS(i-1, j), LCS(i, j-1))$ altrimenti.

Infatti, come base della definizione, la LCS di due sequenze di cui una è vuota risulta vuota. Il passo induttivo corrisponde al caso in cui entrambi i prefissi sono non vuoti (si noti che in questo caso la dimensione del problema è una coppia di numeri e consideriamo l'ordinamento prodotto). Supponendo che le tre sequenze $LCS(i, j-1)$, $LCS(i-1, j)$, $LCS(i-1, j-1)$ siano note, vogliamo calcolare $LCS(i, j)$. Occorre esaminare $X(i)$ e $Y(j)$ e considerare due casi.

- Se $X(i) = Y(j)$, tutte le sottosequenze comuni di $X[1..i]$ e $Y[1..j]$ sono tutte quelle di $X[1..i-1]$ e $Y[1..j-1]$, e tutte quelle ottenute aggiungendo in fondo a una di queste la coppia (i, j). È quindi facile vedere che $LCS(i, j) = LCS(i-1, j-1) \cdot (i, j)$.

- Se $X(i) \neq Y(j)$, le sottosequenze comuni di $X[1..i]$ e $Y[1..j]$ non possono includere la coppia (i, j). Sono quindi sottosequenze comuni di $X[1..i-1]$ e $Y[1..j]$, oppure sottosequenze comuni di $X[1..i]$ e $Y[1..j-1]$, quindi $LCS(i, j)$ è la più lunga fra le due sequenze $LCS(i-1, j)$ e $LCS(i, j-1)$.

La relazione di ricorrenza, considerando il caso peggiore, è la seguente:

$$T(n, m) = T(n-1, m) + T(n, m-1) + 1$$

Considerando la somma $k = n + m$ si ha $T(k) = 2\,T(k-1) + 1$, che è la relazione di ricorrenza delle torri di Hanoi. Si ha quindi un algoritmo esponenziale, si può dare un algoritmo migliore con la tecnica della programmazione dinamica. Infatti, dato che ogni problema di dimensione inferiore è utilizzato nella risoluzione di più problemi di dimensione superiore, è meglio per efficienza procedere bottom-up anziché top-down, memorizzando via via le soluzioni dei problemi.

Si costruisce una matrice `LCS` con m+1 righe ed n+1 colonne. In base alla definizione induttiva data sopra, la prima riga e la prima colonna vanno riempite con la sequenza vuota, di lunghezza 0. Si può poi procedere riga per riga (o colonna per colonna): l'ultima casella, cioè quella nell'angolo a destra in basso, conterrà la soluzione. In ogni casella non occorre memorizzare tutta la LCS ma basta mettere:

- se si ha lo stesso carattere sulla riga e sulla colonna, una "freccia diagonale", aumentando di 1 la lunghezza rispetto alla casella puntata dalla freccia;

- se sulla riga e sulla colonna ci sono due caratteri diversi, una freccia verso la cella di lunghezza maggiore fra la contigua sopra e la contigua a sinistra (se hanno uguale lunghezza si sceglie per esempio quella sopra).

La lunghezza è la stessa di quella della cella puntata dalla freccia. La LCS corrispondente a ciascuna casella si ottiene, dall'ultimo elemento al primo, seguendo le frecce a partire dall'ultima casella. Si noti che non è quindi necessario memorizzare i caratteri nelle celle, mentre la freccia è necessaria per ricostruire all'indietro la LCS, e la lunghezza è necessaria per calcolare le successive celle contigue. Se si è interessati solo alla lunghezza della LCS, ma non alla effettiva sequenza, le frecce non servono, e basta costruire la matrice delle lunghezze.

Vediamo un esempio:

		A	T	C	B	A	B
	0	0	0	0	0	0	0
B	0	0↑	0↑	0↑	↖ 1	← 1	↖ 1
A	0	↖ 1	← 1	← 1	↑ 1	↖ 2	← 2
C	0	↑ 1	↑ 1	↖ 2	← 2	↑ 2	↑ 2
A	0	↖ 1	↑ 1	↑ 2	↑ 2	↖ 3	← 3
T	0	↑ 1	↖ 2	↑ 2	↑ 2	↑ 3	↑ 3
B	0	↑ 1	↑ 2	↑ 2	↖ 3	↑ 3	↖ 4
A	0	↖ 1	↑ 2	↑ 2	↑ 3	↖ 4	↑ 4

Abbiamo evidenziato le caselle corrispondenti alla LCS `ACAB`, di lunghezza 4, che può essere ricostruita a partire dall'ultima casella, e a un'altra LCS `ACBA`.

Si noti che per calcolare `LCS(i,j)` basta conoscere tre caselle contigue (`LCS(i-1,j-1)`, `LCS(i-1,j)`, `LCS(i,j-1)`) e viceversa ogni casella può essere utilizzata per calcolarne altre tre.

Implementazione: come già detto, se si memorizza l'intera matrice, non è necessario inserire in `i,j` l'intera sequenza $LCS(i,j)$: basta memorizzarne solo la lunghezza e il riferimento alla LCS utilizzata per trovarla, lo spazio necessario è quindi $O(mn)$. Usiamo per semplicità due matrici: la matrice `L` delle lunghezze (interi) e la matrice `R` dei riferimenti (coppie di indici, oppure tre valori convenzionali).

```
for (i = 0; i <= m; i++) L[i,0] = 0
for (j = 0; j <= n; j++) L[0,j] = 0

for (i = 1; i <= m; i++)
  for (j = 1; j <= n; j++)
    if (X[i] = Y[j])
      L[i,j] = L[i-1,j-1] + 1; R[i,j] = ↖
    else if (L[i,j-1] > L[i-1,j])
        L[i,j] = L[i,j-1; R[i,j] = ←
    else L[i,j] = L[i-1,j; R[i,j] = ↑
```

Come già osservato, per ricostruire la LCS si parte da `(m,n)` e si va all'indietro seguendo le frecce, in corrispondenza di ogni freccia diagonale si ha un elemento della sequenza (scrivere l'algoritmo per esercizio).

La complessità temporale è $T(m,n) = \Theta(mn)$ (costruzione di matrici $m \times n$). Assumendo come al solito $m \sim n$, l'algoritmo è quadratico: molto meglio dell'algoritmo ingenuo esponenziale.

La complessità spaziale è anch'essa $S(m,n) = \Theta(mn)$ (memorizzazione di matrici $m \times n$). Ottimizzazioni dello spazio: la prima riga e la prima colonna della matrice `R` non contengono alcuna informazione, si potrebbe quindi in questo caso usare una matrice $m \times n$. Si è preferito usare una matrice $m+1 \times n+1$ per avere gli stessi indici della matrice `L`. Alternativamente, ma complicando un po' il codice, si potrebbe usare anche per `L` una matrice $m \times n$, poiché la prima riga e la prima colonna sono sempre costituite da tutti zeri. Infine, se serve solo la lunghezza, non occorre conservare l'intera matrice ma in ogni momento solo una porzione: $\Theta(m+n)$ o anche $\Theta(\min(m,n))$ (verificare per esercizio). Con queste ottimizzazioni si ottiene quindi una complessità spaziale lineare.

2.1.3 Elementi costitutivi della programmazione dinamica

La programmazione dinamica si può applicare ad alcuni problemi che sono scomponibili in sottoproblemi simili tra loro, ma non a tutti. Per applicare la programmazione dinamica, il problema in esame deve avere due caratteristiche:

- L'esistenza di una *sottostruttura ottima*: la soluzione del problema deve contenere le soluzioni dei sottoproblemi necessari alla sua soluzione. Negli esempi visti:
  - Se consideriamo gli algoritmi che forniscono la sequenza di *tutti* i numeri di Fibonacci fino al valore fornito in input (contenuto dell'array `fib`) e, analogamente, la tabella di *tutti* i coefficienti binomiali "più piccoli" di quello richiesto in input (ossia, il contenuto del triangolo di Tartaglia riempito fino alla casella corrispondente alla coppia in input), è chiaro che la soluzione per un dato input contiene anche le soluzioni per gli input "più piccoli", che definiscono i sottproblemi in cui si scompone il problema principale.
  - LCS è un *problema di ottimizzazione*: tra molte scelte possibili – che rispettano vincoli definiti dal problema – chiede di farne una ottimale rispetto a uno specifico criterio, nel caso specifico la sequenza più lunga. Si considerano in input le sequenze $X[1..m]$ e $Y[1..m]$ e si risolvono come sottoproblemi istanze dello stesso problema su tutte le sequenze